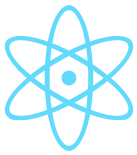
{ AlgoTutor }



Top 20 React Interview Questions & Answers

Ace Your Frontend Interviews with Confidence

2026 Premium Edition • Interview Handbook

Introduction

What is React?

React is a premier JavaScript library for building user interfaces. Maintained by Meta, it powers the frontend of modern web applications through a declarative, component-based architecture.

Why Interviews Matter

React is the most highly demanded frontend skill globally. Top tech companies evaluate not just your ability to write code, but your architectural understanding, performance optimization skills, and mastery of modern Hooks.

Skills Interviewers Evaluate

- **Core Concepts:** Virtual DOM, Reconciliation, JSX, and Unidirectional Data Flow.
- **Hooks Mastery:** In-depth knowledge of useState, useEffect, useMemo, useCallback, and custom hooks.
- **Architecture & State:** Prop drilling, Context API, Redux/Zustand, and component composition.
- **Performance Optimization:** Lazy loading, memoization, and rendering optimization.

Top 20 React Interview Questions

Q1: What is React and how does it work?

Beginner

Concepts: Component-Based Architecture, Virtual DOM, Unidirectional Data Flow.

Why Interviewers Ask It: To assess fundamental understanding of the library and its core philosophy.

Answer: React is an open-source, component-based front-end library responsible only for the view layer of the application. It works by maintaining a Virtual DOM. When state changes, React updates the Virtual DOM, diffs it against the previous version, and optimally updates the real DOM.

Real-World Analogy: *Think of it like updating a ledger. Instead of rewriting the whole book (Real DOM) for a new entry, you write it on a scratchpad (Virtual DOM), see what changed, and only update that specific line.*

```
const App = () =>



;
```

Best Practice:
Keep components small and focused on a single responsibility.

Common Mistake:
Confusing React (a library) with Angular (a full framework).

💡 **Interview Tip:** Always mention 'Virtual DOM' and 'Component reusability'.

Q2: What is JSX?

Beginner

Concepts: Babel Compilation, Syntax Sugar, React.createElement.

Why Interviewers Ask It: JSX is the standard way to write React. Interviewers want to ensure you know what happens under the hood.

Answer: JSX stands for JavaScript XML. It allows us to write HTML inside JavaScript. Under the hood, Babel compiles JSX into React.createElement() calls. It makes code easier to read and write.

Real-World Analogy: *Like writing shorthand notes that an assistant (Babel) later translates into proper formal letters (JavaScript objects).*

```
// JSX
const element =



;

// Compiled
const element = React.createElement('h1', {className: 'title'},
'Hello');
```

Best Practice:
Always use camelCase for HTML attributes (e.g., className, onClick).

Common Mistake:
Forgetting that JSX must return a single parent element (or Fragment).

💡 **Interview Tip:** Explain that JSX prevents XSS attacks by escaping values by default.

Q3: What is the Virtual DOM and why is it fast?

Intermediate

Concepts: Reconciliation, Diffing Algorithm, Batch Updates.

Why Interviewers Ask It: This is a core performance concept. You must know how React optimizes rendering.

Answer: The Virtual DOM is a lightweight JavaScript copy of the Real DOM. It is fast because updating a JS object is computationally cheaper than repainting the browser's Real DOM. React uses a diffing algorithm to compare the new Virtual DOM with the old one and only applies the exact differences (Reconciliation).

Real-World Analogy: *Like a rough draft of an architectural blueprint. You make changes to the draft first, and only hammer nails (Real DOM) once the draft is finalized.*

```
No specific code, conceptual understanding is key.
```

Best Practice:
Provide unique 'key' props to lists to help the diffing algorithm.

Common Mistake:
Assuming Virtual DOM is a React-only concept (Vue uses it too) or that it is faster than vanilla JS DOM operations (it's faster than NAIVE DOM operations).

💡 **Interview Tip:** Use the word 'Reconciliation' to sound experienced.

Q4: Explain Props vs State.

Beginner

Concepts: Immutability, Component Lifecycle, Data Flow.

Why Interviewers Ask It: Fundamental difference in how data is managed and passed in React.

Answer: Props (Properties) are used to pass data from a parent component to a child component; they are read-only (immutable). State is a local data storage that is local to the component only and cannot be passed to other components directly; it is mutable and triggers a re-render when changed.

Real-World Analogy: *Props are like your DNA (passed from parents, cannot change). State is like your mood (internal, changes over time based on events).*

```
const Child = ({ name }) =>
{name}
; // Prop
const [count, setCount] = useState(0); // State
```

Best Practice:
Keep state as high as necessary (lifting state up) and use props for passing it down.

Common Mistake:
Attempting to modify props directly inside a child component.

💡 **Interview Tip:** Create a clear table mentally: Props = External/Immutable, State = Internal/Mutable.

Q5: What are React Hooks?

Beginner | **Concepts: Functional Components, State Management, Side Effects.**

Why Interviewers Ask It: Hooks are the modern standard for React. Knowing them is non-negotiable.

Answer: Hooks are functions that let you 'hook into' React state and lifecycle features from function components. They were introduced in React 16.8 to allow functional components to have state and side effects, essentially replacing class components.

Real-World Analogy: *Like adding power-ups to a standard character (functional component) giving it the abilities of a boss character (class component).*

```
import { useState, useEffect } from 'react';
```

Best Practice:
Only call Hooks at the top level. Don't call them inside loops, conditions, or nested functions.

Common Mistake:
Using hooks inside class components (they will throw an error).

💡 **Interview Tip:** Mention the Rules of Hooks.

Q6: How does useState work?

Beginner

Concepts: Destructuring, Asynchronous Updates, Re-rendering.

Why Interviewers Ask It: The most common hook. Tests basic state manipulation.

Answer: useState is a hook that lets you add React state to function components. It returns an array with two values: the current state and a function to update it. When the update function is called, React re-renders the component.

Real-World Analogy: *Like a scoreboard in a game. The score is the state, and the referee's remote is the setter function.*

```
const [count, setCount] = useState(0);
setCount(prev => prev + 1); // Best practice for previous state
```

Best Practice:
Use functional updates (prev => prev + 1) when the new state depends on the old state.

Common Mistake:
Mutating state directly (e.g., state.count = 1) instead of using the setter function.

💡 **Interview Tip:** Explain that state updates are batched for performance.

Q7: Explain the useEffect hook.

Intermediate

Concepts: Side Effects, Dependency Array, Cleanup Function, Component Lifecycle.

Why Interviewers Ask It: Crucial for handling side effects like API calls and DOM manipulation.

Answer: useEffect lets you perform side effects in function components. It takes two arguments: a callback function and a dependency array. It runs after the first render and after every update where a dependency has changed. Returning a function from the callback handles cleanup.

Real-World Analogy: *Like a subscription. You subscribe when you enter a room (mount), read the magazine (effect), and unsubscribe when you leave (cleanup).*

```
useEffect(() => {
  const timer = setInterval(() => console.log('Tick'), 1000);
  return () => clearInterval(timer); // Cleanup
}, []); // Empty array = runs once on mount
```

Best Practice:
Always clean up subscriptions and timers to prevent memory leaks.

Common Mistake:
Forgetting the dependency array, causing infinite re-renders.

💡 **Interview Tip:** Compare it to componentDidMount, componentDidUpdate, and componentWillUnmount.

Q8: What is the difference between useMemo and useCallback?

Advanced

Concepts: Memoization, Referential Equality, Performance Optimization.

Why Interviewers Ask It: Tests advanced performance optimization knowledge.

Answer: Both are used for performance optimization by caching values. useMemo memoizes the resulting *value* of a calculation. useCallback memoizes the *function definition* itself. Use useMemo to avoid expensive calculations on every render, and useCallback to avoid recreating functions passed as props to child components.

Real-World Analogy: *useMemo is like saving the answer to a complex math problem. useCallback is like saving the formula itself so you don't have to rewrite it.*

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a,
b]);
const memoizedCallback = useCallback(() => doSomething(a, b), [a, b]);
```

Best Practice:
Don't overuse them. The cost of memoization can sometimes be higher than the re-render.

Common Mistake:
Using them everywhere 'just in case', which degrades performance.

💡 **Interview Tip:** Mention 'referential equality' when explaining useCallback.

Q9: What is useRef used for?

Intermediate

Concepts: Mutable Instance Variables, DOM Access, Re-renders.

Why Interviewers Ask It: Tests knowledge of accessing DOM elements directly and preserving values without re-rendering.

Answer: useRef returns a mutable ref object whose .current property is initialized to the passed argument. It is used for two main purposes: 1) Accessing a DOM element directly. 2) Storing a mutable value that does NOT trigger a re-render when updated (unlike useState).

Real-World Analogy: *Like a safe in your house. You can change what's inside the safe (the value), but doing so doesn't require rebuilding the house (re-rendering).*

```
const inputRef = useRef(null);
const focusInput = () => inputRef.current.focus();
```

Best Practice:
Use it for managing focus, text selection, or media playback.

Common Mistake:
Using refs for things that can be done declaratively with state.

💡 **Interview Tip:** Contrast it with useState regarding re-renders.

Q10: Explain the Context API.

Intermediate

Concepts: Global State, Prop Drilling, Provider Pattern.

Why Interviewers Ask It: Evaluates your ability to manage global state without external libraries like Redux.

Answer: The Context API provides a way to pass data through the component tree without having to pass props down manually at every level (prop drilling). It consists of React.createContext, a Provider component, and a Consumer (or useContext hook).

Real-World Analogy: *Like a PA system in a school. Instead of passing a message from teacher to student to student, you just broadcast it to anyone listening.*

```
const ThemeContext = createContext('light');
// Wrap app in
// Child: const theme = useContext(ThemeContext);
```

Best Practice:
Use it for low-frequency updates like theme, locale, or authentication status.

Common Mistake:
Using Context for highly dynamic state, which causes excessive re-rendering of all consumers.

💡 **Interview Tip:** Mention that Redux is better for complex, high-frequency state updates.

Q11: What is Prop Drilling and how do you avoid it?

Beginner

Concepts: Data Flow, Component Composition, Global State.

Why Interviewers Ask It: Common architectural challenge in React.

Answer: Prop drilling is the process of passing data from a top-level component down to deeply nested components via props, even if the intermediate components don't need the data. It can be avoided using the Context API, state management libraries (Redux, Zustand), or component composition.

Real-World Analogy: *Like passing a package through 10 middlemen to reach the final person. It's inefficient.*

```
No code needed. Focus on architectural explanation.
```

Best Practice:
Try Component Composition (passing components as props) before jumping to Context.

Common Mistake:
Immediately reaching for Redux just to avoid one level of drilling.

💡 **Interview Tip:** Mentioning 'Component Composition' is a huge bonus point.

Q12: How does React Router work?

Intermediate

Concepts: Client-side Routing, SPAs, BrowserRouter, Route, Link.

Why Interviewers Ask It: Standard library for routing in SPAs.

Answer: React Router enables client-side routing in React apps. Instead of making a request to the server for a new HTML page, React Router intercepts the URL change and renders the appropriate React component locally, keeping the app a Single Page Application (SPA).

Real-World Analogy: *Like a magical doorway in a room. Depending on the sign on the door (URL), the room behind it instantly changes without you having to walk to a new building.*

```
} />
```

Best Practice: Use instead of tags to prevent full page reloads.

Common Mistake:
Using traditional href attributes which break the SPA experience.

💡 **Interview Tip:** Mention recent additions like React Router v6 loaders/ actions.

Q13: Controlled vs Uncontrolled Components?

Intermediate

Concepts: Forms, Two-way Data Binding (conceptually), Refs vs State.

Why Interviewers Ask It: Form handling is a critical part of frontend development.

Answer: In a Controlled Component, form data is handled by the React component's state (using useState). In an Uncontrolled Component, form data is handled by the DOM itself (using useRef to extract values). Controlled is generally preferred for immediate validation.

Real-World Analogy: *Controlled: A strict teacher grading every letter you write instantly. Uncontrolled: A teacher who only grades the final submitted paper.*

```
// Controlled
{name} setName(e.target.value)} />
// Uncontrolled
```

Best Practice:
Use controlled components for complex forms needing dynamic validation.

Common Mistake:
Mixing both paradigms on the same input element.

💡 **Interview Tip:** Note that File inputs are always uncontrolled in React.

Q14: What is React.memo()?

Intermediate

Concepts: Higher Order Components, Shallow Comparison, Memoization.

Why Interviewers Ask It: Another key performance optimization technique.

Answer: React.memo is a Higher Order Component (HOC). It wraps a functional component and prevents it from re-rendering if its props have not changed. It does a shallow comparison of previous and new props.

Real-World Analogy: *Like a bouncer at a club. If you look exactly the same as 5 minutes ago (same props), they don't bother checking your ID again (no re-render).*

```
const MyComponent = React.memo(function MyComponent(props) {
  return
{props.data}
;
});
```

Best Practice:
Use it on components that render often with the same props.

Common Mistake:
Wrapping every component in React.memo, which adds unnecessary shallow comparison overhead.

💡 **Interview Tip:** Explain how it pairs with useCallback (passing new function references breaks memoization without useCallback).

Q15: What is Lazy Loading and Code Splitting?

Advanced

Concepts: Webpack, Bundling, React.lazy, Suspense, Performance.

Why Interviewers Ask It: Tests knowledge of optimizing bundle size and initial load time.

Answer: Code Splitting is breaking the app bundle into smaller chunks. Lazy Loading is delaying the loading of these chunks until they are actually needed (e.g., when a user navigates to a specific route). This drastically reduces the initial load time. In React, this is done using React.lazy() and Suspense.

Real-World Analogy: *Like buying furniture for a house. You don't buy everything at once; you buy the living room set first, and buy the guest bed only when a guest arrives.*

```
const LazyComponent = React.lazy(() => import('./LazyComponent'));

Loading...
```

}>

Best Practice:
Implement code splitting at the route level for the biggest performance wins.

Common Mistake:
Forgetting the Suspense boundary, which crashes the app.

💡 **Interview Tip:** Mention Webpack or Vite under the hood.

Q16: Explain 'Lifting State Up'.

Beginner

Concepts: State Management, Sibling Communication, Top-down Data Flow.

Why Interviewers Ask It: Basic architectural pattern in React.

Answer: When several components need to share the same changing data, you should elevate the state to their closest common ancestor. The parent passes the state and the updater function down to the children via props.

Real-World Analogy: *If two sibling managers need to coordinate, they don't talk directly; they report to their mutual boss, who then dictates the strategy to both.*

```
No code needed. Focus on the concept of common ancestor.
```

Best Practice:
Keep state as low as possible, but lift it when siblings need to share it.

Common Mistake:
Lifting state too high (e.g., to the root App component) unnecessarily.

💡 **Interview Tip:** Draw a mental tree diagram to explain the closest common ancestor.

Q17: What are Custom Hooks?

Advanced

Concepts: Reusability, Separation of Concerns, Logic Extraction.

Why Interviewers Ask It: Shows you can write DRY, reusable, and modular React code.

Answer: Custom hooks are standard JavaScript functions whose name starts with 'use' and that call other React Hooks. They allow you to extract component logic into reusable functions.

Real-World Analogy: *Like creating a macro on your keyboard. You combine several actions (useState, useEffect) into one single button press (useMyHook).*

```
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);
  useEffect(() => { /* event listener */ }, []);
  return width;
}
```

Best Practice:
Always start the name with 'use' so React linting rules apply.

Common Mistake:
Trying to share state *values* between components using a custom hook (custom hooks share *logic*, not state).

💡 **Interview Tip:** Provide examples like useFetch, useWindowSize, or useAuth.

Q18: What are Error Boundaries?

Intermediate

Concepts: Fallback UI, ComponentDidCatch, Error Handling.

Why Interviewers Ask It: Evaluates knowledge of production-level robustness.

Answer: Error boundaries are React components that catch JavaScript errors anywhere in their child component tree, log those errors, and display a fallback UI instead of crashing the whole component tree. They only catch errors in the render phase, lifecycle methods, and constructors.

Real-World Analogy: *Like a circuit breaker in a house. If a short circuit happens in a bedroom, the breaker trips to contain it, instead of burning the whole house down.*

```
class ErrorBoundary extends React.Component {
  componentDidCatch(error, info) { logError(error); }
  render() { return this.state.hasError ?  : this.props.children; }
}
```

Best Practice:
Wrap top-level routes or major UI sections in boundaries.

Common Mistake:
Assuming error boundaries catch errors in event handlers or async code (they don't).

💡 **Interview Tip:** Note that currently, Error Boundaries must be written as Class Components (or use libraries like react-error-boundary).

Q19: How does React's diffing algorithm work?

Advanced

Concepts: Heuristic Algorithm, O(n) Complexity, Key Props.

Why Interviewers Ask It: Deep dive into React internals.

Answer: React diffs two Virtual DOM trees by making two assumptions to achieve O(n) complexity: 1) Two elements of different types will produce different trees (React tears down the old tree completely). 2) The developer can hint at which child elements may be stable across different renders with a 'key' prop.

Real-World Analogy: *Like comparing two lists of inventory. If the shelf type changed, you just replace the shelf. If the items shifted, you use barcode IDs (keys) to track them efficiently.*

```
    {items.map(item =>
• {item.name}
  )}
```

Best Practice:
Always use unique, stable IDs for keys in lists.

Common Mistake:
Using array index as a key for lists that can be reordered or filtered.

💡 **Interview Tip:** Knowing the time complexity (from O(n^3) theoretically to O(n) practically) is a massive flex.

Q20: What is React Strict Mode?

Beginner

Concepts: Development Tools, Deprecation Warnings, Side Effect Detection.

Why Interviewers Ask It: Shows awareness of React development tools.

Answer: StrictMode is a tool for highlighting potential problems in an application. It activates additional checks and warnings for its descendants. It only runs in development and does not impact production build. Notably, it double-invokes certain lifecycle methods and hooks to detect side effects.

Real-World Analogy: *Like a drill sergeant in training camp. They make you do things twice as hard to catch weaknesses before you go to the real battlefield (production).*

Best Practice:
Keep it enabled during development to catch bugs early.

Common Mistake:
Thinking the double console.log() in useEffect is a bug; it's a feature of StrictMode.

💡 **Interview Tip:** Explain the double-invocation behavior as that trips up many beginners.

React Hooks Cheat Sheet

Hook	Primary Use Case	Triggers Re-render?
`useState`	Local state management within a component.	Yes
`useEffect`	Side effects (data fetching, subscriptions, DOM manipulation).	No (but changing state inside it will)
`useContext`	Accessing global state without prop drilling.	Yes (if context value changes)
`useRef`	Accessing DOM elements or storing mutable values.	No
`useMemo`	Caching expensive calculations to avoid recalculation.	No
`useCallback`	Caching function definitions to prevent unnecessary child renders.	No

Performance Optimization Checklist

- ✓ **Use React.memo:** Wrap components that receive stable props.
- ✓ **Implement Code Splitting:** Use `React.lazy()` and Suspense for route-level splitting.
- ✓ **Optimize Dependency Arrays:** Ensure `useEffect` dependencies are completely accurate.
- ✓ **Use useCallback & useMemo Wisely:** Only apply them when passing props down to memoized child components or performing highly expensive loops.
- ✓ **Key Prop Strategy:** Never use array index as a key for dynamic lists. Use unique IDs.
- ✓ **Debounce API Calls:** Limit the rate of state updates for search inputs using custom debounce hooks.

Final Interview Preparation Checklist

- ✓ Revise all Hooks (Rules & Syntax)
- ✓ Practice Coding (Build small components)
- ✓ Build 1-2 Fullstack Projects
- ✓ Understand Global State (Context/Redux)
- ✓ Revise React Router (v6 specifically)
- ✓ Optimize Performance (Memoization)
- ✓ Explain Concepts Clearly (Rubber Duck Method)
- ✓ Solve Mock Interview Questions

Thank You!

Best of luck with your React interviews. You've got this.

{ AlgoTutor }

Learn • Build • Grow

Follow AlgoTutor on LinkedIn for more technical resources.